IT SECURITY MANAGEMENT & DEVOPS · INDIVIDUAL PROJECT

ITSM Ticket Management System

Containerization, Kubernetes deployment and observability of a Django ticket application

Kaushal Patidar · Student ID 10008313 · Academic Year 2025–2026

Executive summary

This project rewrites a ticket CRUD service as a Django application delivered with MariaDB, Gunicorn, Docker, Docker Compose and Minikube Kubernetes. It supports authenticated ticket creation, listing and search, editing and deletion, plus a public health endpoint for probes. Evidence documents the browser workflow, database persistence, Kubernetes deployment, a 500 virtual-user load test and automatic container recovery.

20,028	321.5	0%	1.20 s
Requests · 500 VUs · 60 s	Requests / second	Failed requests	p95 latency

Outcome: All checks passed and the application remained running, but p95 exceeded the 1-second target. CPU saturation and throttling indicate a CPU-bound bottleneck at this load.

Solution overview

Area	Implementation / evidence
Application and data	Django MVT with authentication, ticket forms and search. MariaDB 11.4 stores relational records through Django ORM. Tickets include category, priority and status.
Packaging	Multi-stage Python 3.12 container runs Gunicorn as a non-root user. Compose connects web and database services with health checks and persistent database storage.
Orchestration	Minikube hosts a namespaced web Deployment and MariaDB StatefulSet with probes, resource requests and limits, persistent volume storage and an HPA.
Validation	Create, list/search, update and delete workflow checked against Kubernetes and database state. Health endpoint is used by Kubernetes probes.

Performance findings

Measure	200 VUs · host port-forward	500 VUs · in cluster	Interpretation
Requests / throughput	15,534 / 253.5 req/s	20,028 / 321.5 req/s	Throughput approaches a plateau.
Failed requests / checks	0% / 100%	0% / 100%	All measured checks passed.
Average / p95 latency	281 ms / 478 ms	1.02 s / 1.20 s	500-VU p95 crossed the 1-second threshold (k6 exit 99).
CPU / memory	Web pod near 1 CPU limit	About 88 MiB / 1 GiB	CPU-bound; memory was not limiting.

Resource analysis. During the 500-VU window, cgroup counters show about 61.5 CPU-seconds used and 77.3 seconds throttled; 79.3% of scheduling periods recorded throttling. At roughly 3.07 ms CPU per request, one CPU core supports throughput close to the measured level. No pod restart or memory-pressure failure was reported after the run.

Operational observations and readiness

Observed	Limitations and follow-up
Kubernetes restarted the web container after its process exited; it returned to Running and Ready about one second later. HPA scaled from one replica to five and later scaled back after load ended.	The 200-VU host and 500-VU in-cluster measurements use different network paths. Load testing covered public login and health pages; database writes were not load-tested.
kubectl, metrics-server, cgroup CPU counters, application logs and k6 summaries supported operational review.	Time-series latency output and Prometheus/Grafana monitoring were not configured. Repeat comparable runs after improving CPU capacity or workload efficiency, and verify scaling while traffic is active.

Security considerations

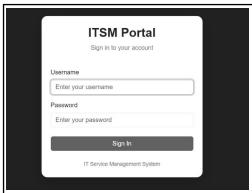
Positive controls include a non-root application user, slim runtime image, database not published to the host, secret references for database credentials, authentication on ticket views and resource limits. The reference report identifies development-grade settings such as DEBUG=True and a placeholder secret key; production deployment requires secure settings and managed secrets.

Report basis

Prepared from the supplied ITSM & DevOps reference report, repository README and project screenshots. Performance figures and operational findings are reproduced as documented in those materials; deployment behavior can vary by environment. The screenshot appendix contains original captures in grayscale with source filenames only.

Screenshot appendix

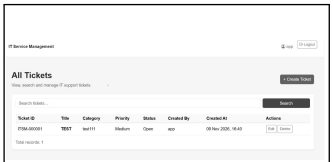
Original project captures shown together as visual evidence. Captions identify source filenames only.



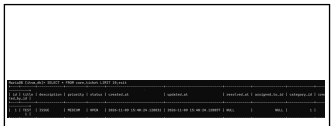
1.1.png



1.png



2.png



3.1.png



3.png



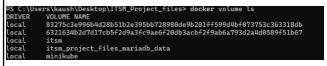
7.png



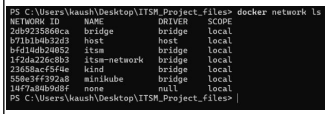
8.png



9.png



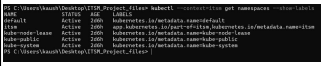
10.png



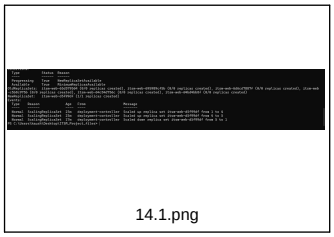
11.png



12.png



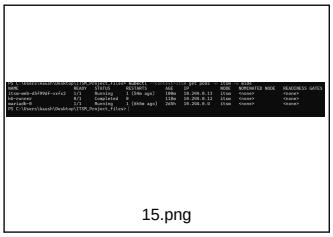
13.png



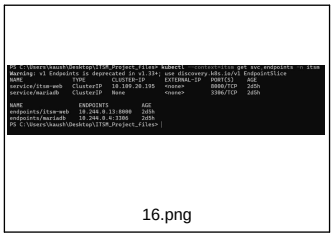
14.1.png



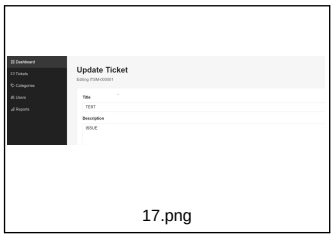
14.png



15.png



16.png



17.png



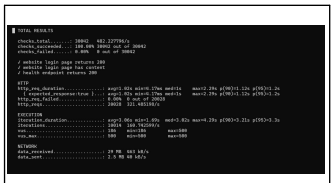
18.png



19.1.png



19.png



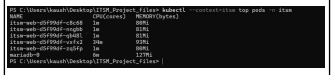
20 (1).png



20 (2).png



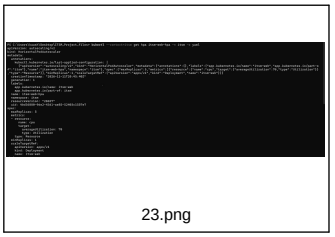
21.png



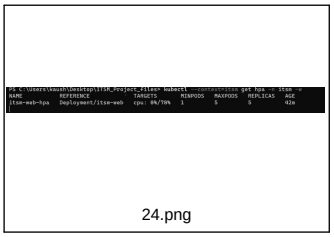
22.png



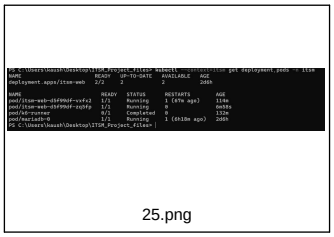
23.1.png



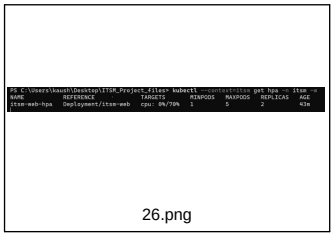
23.png



24.png



25.png



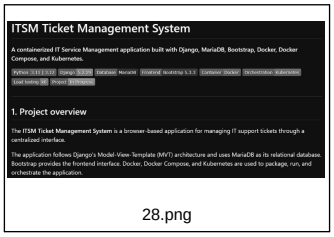
26.png



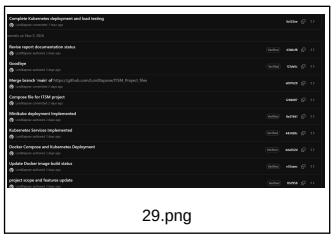
27.1.png



27.png



28.png



29.png



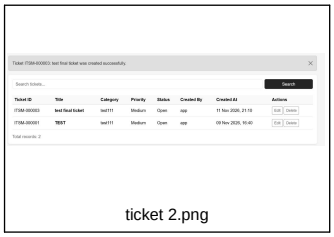
database verify.png



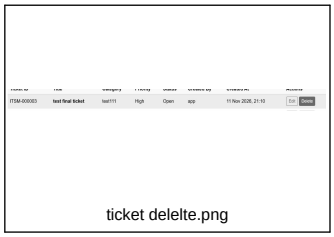
delete ticketr 2.png



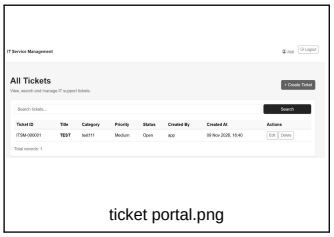
deleted ticket .png



ticket 2.png



ticket delelte.png



ticket portal.png



ticket update.png